🍄 Mushroom Hunt — первые задачи: оживляем меню

Привет! Это гайд по первым задачам проекта. Каждая задача живёт **в своём файле** — ты не мешаешь товарищам, а мержи проходят безболезненно.

Как устроен проект (2 минуты)

- `Main.kt` — «директор» игры. Держит скин (стили) и решает, какой экран сейчас на экране :)
- `MainMenuScreen.kt` — главное меню: фон + столбик кнопок. **Общий файл, не трогаем.**
- `GameButton.kt` — наша кнопка. У неё есть свойство `onClick` — «что сделать при нажатии». **Общий файл, не трогаем.**
- `InfoPopup.kt` — всплывающее окошко-заглушка. Можно подсматривать, как сделано, но не ломать.
- `MenuAction.kt` — интерфейс-контракт: у каждого действия меню есть метод `run()`.
- `PlayAction.kt`, `RulesAction.kt`, `SettingsAction.kt`, `ExitAction.kt` — **вот тут работаешь ты.** Каждый файл = одна кнопка меню.

Сейчас нажатие любой кнопки показывает заглушку:

```
override fun run() {
    stage.addActor(InfoPopup("PLAY — IN DEVELOPMENT", game.skin))
}
```

Твоя задача — заменить заглушку на настоящее поведение.

Задача 1 — EXIT: научить игру закрываться 🔌

Файл: `ExitAction.kt` · **Сложность:** ⭐

libGDX умеет закрывать приложение сам, одной строчкой — и это работает **и на десктопе, и на Android**:

```
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.Gdx
import com.badlogic.gdx.scenes.scene2d.Stage

class ExitAction(private val game: Main, private val stage: Stage) : MenuAction {

    override fun run() {
        // Просим libGDX завершить приложение корректно:
```

```
            // на десктопе закроется окно, на Android приложение свернётся с finish().
            Gdx.app.exit()
        }
    }
```

Почему `Gdx.app.exit()`, а не `System.exit(0)`? `Gdx.app.exit()` сначала вызывает у игры `dispose()` — все текстуры и ресурсы освободятся аккуратно. `System.exit()` бьёт по приложению сразу, видеопамять может утечь, а на Android это вообще не по правилам.

Бонус (по желанию): сделай красивее — сначала попап «Точно выйти?» с кнопками ДА/НЕТ. ДА зовёт `Gdx.app.exit()`, НЕТ зовёт `remove()` у попапа. Подглядывай устройство попапа в `InfoPopup.kt`.

Проверка: запустил, нажал EXIT — окно закрылось. Всё, ты сделал первую фичу 🎉

Задача 2 — SETTINGS: меню настроек (пока без функционала) ⚙️

Файл: `SettingsAction.kt` (+ новый файл `SettingsPopup.kt` для порядка) · **Сложность:** ⭐⭐

Задача — **визуал**: надписи, переключатели, ползунок. Настоящая логика (реально глушить звук) подключим позже. В нашем `uiskin` уже есть стили для `CheckBox`, `Slider` и `SelectBox` — ничего рисовать не надо.

Создай файл `SettingsPopup.kt` рядом с остальными (каркас скопирован из `InfoPopup.kt` — так и задумано, учимся на примере):

```
// SettingsPopup.kt — окошко настроек. Пока только внешний вид.
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.scenes.scene2d.ui.*

class SettingsPopup(skin: Skin) : Table() {

    init {
        setFillParent(true) // занять весь экран (затемнение под окошком)
        background(skin.newDrawable("white", 0f, 0f, 0f, 0.6f)) // полупрозрачная тень

        // === ВИДЖЕТЫ ===

        // Переключатель (чекбокс): включён/выключен. Сам ничего не делает — просто тумблер
        val chkMute = CheckBox("  MUTE", skin)

        // Ползунок громкости: Slider(мин, макс, шаг, вертикальный?, скин)
        val sldVolume = Slider(0f, 100f, 1f, false, skin)
        sldVolume.value = 70f // начальное положение ползунка

        // Переключатель вибрации (для телефонов)
        val chkVibration = CheckBox("  VIBRATION", skin)
```

```
        // Выпадающий список языков
        val selLanguage = SelectBox<String>(skin)
        selLanguage.setItems("RU", "EN", "DE")

        // Кнопка закрытия — наш общий GameButton
        val btnClose = GameButton("CLOSE", skin)
        btnClose.onClick = { remove() } // remove() — «уйти со сцены», окошко исчезнет

        // === РАСКЛАДКА ===

        // Карточка настроек — тёмная коробочка по центру
        val box = Table(skin)
        box.background(skin.newDrawable("white", 0.15f, 0.12f, 0.1f, 0.95f))

        // Заголовок. Label — просто текстовая надпись.
        box.add(Label("SETTINGS", skin)).pad(20f).colspan(2).row()

        // Строка «VOLUME + ползунок»: слева надпись, справа виджет
        box.add(Label("VOLUME", skin)).pad(10f).left()
        box.add(sldVolume).pad(10f).width(220f).row()

        // Строка «MUTE + тумблер»
        box.add(Label("MUTE", skin)).pad(10f).left()
        box.add(chkMute).pad(10f).row()

        // Строка «VIBRATION + тумблер»
        box.add(Label("VIBRATION", skin)).pad(10f).left()
        box.add(chkVibration).pad(10f).row()

        // Строка «LANGUAGE + выпадашка»
        box.add(Label("LANGUAGE", skin)).pad(10f).left()
        box.add(selLanguage).pad(10f).row()

        // Кнопка CLOSE во всю ширину, побольше отступ сверху
        box.add(btnClose).pad(20f).colspan(2).growX().row()

        // Коробочку — в центр экрана
        add(box).expand().center()
    }
}
```

А в `SettingsAction.kt` зови свой попап вместо заглушки:

```
override fun run() {
    stage.addActor(SettingsPopup(game.skin))
}
```

Что тут происходит:

- `colspan(2)` — ячейка занимает две колонки (заголовок и кнопка — на всю ширину).
- `.left()` — прижать к левому краю ячейки, чтобы надписи стояли ровно, как в таблице.
- `.width(220f)` — фиксируем ширину ползунка, иначе он растянется как хочет.

- `Slider(0f, 100f, 1f, false, skin)` — от 0 до 100, шаг 1, горизонтальный.

Идеи для расширения (когда база готова): строка DIFFICULTY с такой же выпадашкой (`EASY/NORMAL/NIGHTMARE`), чекбокс FULLSCREEN, ползунок MUSIC отдельно от SFX.

Проверка: кнопка SETTINGS → появляется окошко, тумблеры щёлкаются, ползунок таскается, язык выбирается, CLOSE всё закрывает.

Задача 3 — RULES: экран правил 📜

Файл: `RulesAction.kt` (+ новый `RulesPopup.kt`) · **Сложность:** ⭐⭐

Текста будет много — значит, учимся скроллить. Заодно сделаем красиво: заголовок, аккуратная карточка, прокрутка.

Сначала — сам текст. Заготовка (смело скопируй текст из обсуждения в ТГ):

```
ЛЕС ПРОСНУЛСЯ

Ты забрёл во владения Лешего. Он предлагает сделку:
...

ПРАВИЛА
1. Игроки ходят по очереди: ты и Леший.
...

Леший никогда не врёт. Но и не говорит всей правды.
```

Теперь код. Трюк длинного текста: `Label` с переносом строк + `ScrollPane` (лента прокрутки):

```
// RulesPopup.kt — экран правил со скроллом.
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.scenes.scene2d.ui.*

class RulesPopup(skin: Skin) : Table() {

    init {
        setFillParent(true)
        background(skin.newDrawable("white", 0f, 0f, 0f, 0.6f))

        // Многострочный текст. \n\n — пустая строка между абзацами.
        val text = "ЛЕС ПРОСНУЛСЯ\n\n" +
                "Ты забрёл во владения Лешего...\n\n" +
                "ПРАВИЛА\n1. ...\n2. ...\n3. ..."

        val label = Label(text, skin)
        label.setWrap(true) // ВАЖНО: разрешить перенос строк. Без этого текст уползёт за э

        // ScrollPane — «окно» с прокруткой: внутри лента, снаружи рамка.
```

```
        // В конструктор — контейнер с текстом и стиль из скина.
        val textTable = Table()
        textTable.pad(10f)
        textTable.add(label).width(520f).left() // фиксированная ширина текста, перенос раб

        val scroll = ScrollPane(textTable, skin)

        val btnClose = GameButton("CLOSE", skin)
        btnClose.onClick = { remove() }

        // Карточка
        val box = Table(skin)
        box.background(skin.newDrawable("white", 0.15f, 0.12f, 0.1f, 0.95f))
        box.add(Label("RULES", skin)).pad(16f).row()        // заголовок
        box.add(scroll).pad(10f).size(560f, 300f).row()     // зона прокрутки 560x300
        box.add(btnClose).pad(16f).row()                    // кнопка внизу

        add(box).expand().center()
    }
}
```

В `RulesAction.kt`:

```
override fun run() {
    stage.addActor(RulesPopup(game.skin))
}
```

Как сделать красиво (чек-лист):

- **Заголовок отделён от тела** — жирная строка сверху, отступ побольше.
- **Фиксированная ширина текста** (`.width(520f)`) — столбик читается, как в книге.
- **Скролл, а не ужатие** — текст никогда не вылезает за карточку.
- **Тёплый тёмный фон карточки** `0.15f, 0.12f, 0.1f` — коричневый «древесный» вайб вместо чистого серого.
- **Пункты правил — с номерами** — глазу за что цепляться.

Идеи: свой заголовок-картинка вместо Label, две страницы (ПРАВИЛА / ЛОР) с кнопками-вкладками.

Проверка: RULES → карточка с текстом, колесо мыши/свайп листает, CLOSE закрывает.

Задача 4 — PLAY: экран-заставка с возвратом в меню 🌲

Файл: новый `PlayScreen.kt` + правка `PlayAction.kt` · **Сложность:** ⭐⭐⭐

Это первая задача про **экран**, а не попап. Отличие: попап — окошко поверх меню, экран — отдельная «сцена» игры. Учимся переключать экраны.

Логика: та же заставка, что в меню (`main-menu1.png`), в уголке кнопка «BACK TO MENU».

```kotlin
// PlayScreen.kt — заготовка игрового экрана. Пока — заставка и возврат в меню.
package ru.triplethall.mushroomhunt

import com.badlogic.gdx.Gdx
import com.badlogic.gdx.ScreenAdapter
import com.badlogic.gdx.graphics.OrthographicCamera
import com.badlogic.gdx.graphics.Texture
import com.badlogic.gdx.graphics.g2d.SpriteBatch
import com.badlogic.gdx.scenes.scene2d.Stage
import com.badlogic.gdx.scenes.scene2d.ui.Table
import com.badlogic.gdx.utils.viewport.FitViewport

class PlayScreen(private val game: Main) : ScreenAdapter() {

    private val camera = OrthographicCamera()
    private val stage = Stage(FitViewport(900f, 450f, camera))
    private val batch = SpriteBatch()

    // Та же картинка, что в меню. Texture занимает видеопамять —
    // два экрана = две копии. Для заставки это ок, позже заведём общий склад ассетов.
    private val bg = Texture(Gdx.files.internal("main-menu1.png"))

    override fun show() {
        Gdx.input.inputProcessor = stage // как в меню: без этого кнопка мёртвая

        val root = Table()
        root.setFillParent(true)
        root.top().left()          // прижать содержимое к верхнему левому углу
        root.pad(16f)              // и отступить от края 16 пикселей
        stage.addActor(root)

        val btnBack = GameButton("BACK TO MENU", game.skin)
        btnBack.onClick = {
            // Переключаем экран обратно на меню...
            game.setScreen(MainMenuScreen(game))
            // ...и прибираем за собой. Кто создал ресурс — тот dispose.
            dispose()
        }
        root.add(btnBack)
    }

    override fun render(delta: Float) {
        batch.begin()
        batch.draw(bg, 0f, 0f, Gdx.graphics.width.toFloat(), Gdx.graphics.height.toFloat())
        batch.end()

        stage.act(delta)
        stage.draw()
    }

    override fun resize(width: Int, height: Int) = stage.viewport.update(width, height, tru

    override fun dispose() {
        stage.dispose()
        bg.dispose()
```

```
            batch.dispose()
        }
    }
```

В `PlayAction.kt` — вместо попапа переключаем экран:

```
override fun run() {
    game.setScreen(PlayScreen(game)) // меню уходит в тень, играет новый экран
}
```

Что тут важно понять:

- `game.setScreen(...)` — единственный способ сменить экран. Игра (Main) просто начинает отдавать кадры другому экрану.
- Старый экран сам себя **не** освободит — поэтому зовём `dispose()` после переключения. Забудешь — получишь утечу видеопамяти.
- `root.top().left()` — так Table прижимает содержимое к углу. Убери — кнопка окажется в центре, как в меню.

Идеи: надпись «ROUND 1» по центру, мигающая подпись «НАЖМИ PLAY ЧТОБЫ НАЧАТЬ», плавное затемнение при переходе (Actions.fadeIn у корневой таблицы).

Проверка: PLAY → заставка с кнопкой в углу → BACK TO MENU → вернулся в меню, всё живое. Повтори цикл раз пять — если игра не падает, утечек нет.

Общие правила гигиены 🧼

1. **Не трогай общие файлы:** `Main.kt`, `MainMenuScreen.kt`, `GameButton.kt`, `InfoPopup.kt`. Работаем только в своих `*Action.kt` и новых файлах.
2. **Коммить часто, github desktop настроен**
3. **Перед пушем** — убедись, что запускается и твоя кнопка работает.
4. **Комментарии пиши как в примерах** — через полгода спасибо скажешь сам себе.
5. Застрял — подсмотри, как соседняя фича сделана. Все четыре задачи — про одни и те же Table/кнопки/попапы, только с разными начинками.

Удачи в лесу. Леший ждёт 🍄